AI · COMPUTER VISION · VIVA PREPARATION

Smart Surveillance System

Complete Technical & Viva-Voce Preparation Report

Stack: Python · OpenCV · YOLOv8 · Flask
Scope: Architecture · Models · ML Metrics · Flask

Smart Surveillance System — In-Depth Technical, Theoretical & Mathematical Report

> A complete analysis of the `smart_surviance` project: its architecture, every module, the computer-vision and machine-learning models it uses, and the mathematics that underpins each feature. This document is written so that it can serve both as engineering documentation and as a defense/explanation document (e.g. for a project report or viva).

Table of Contents

---

1. Executive Summary

The **Smart Surveillance System** is a modular, real-time computer-vision platform written in Python. It ingests a video stream (webcam, IP camera, or file), detects and tracks objects with a deep neural network (YOLOv8), and layers a large suite of higher-level analytics on top: facial recognition, behavior/pose analysis, person re-identification, fall/loitering/abandoned-object detection, crowd density estimation, fire/smoke, weapon, violence, PPE, and license-plate recognition. Events are de-duplicated and throttled into a standardized alert stream, persisted into SQLite databases, and surfaced live through a Flask + Socket.IO web dashboard.

The system blends two paradigms:

- **Learned models** (deep CNNs): YOLOv8 for object detection, YuNet/SFace or dlib for faces, MediaPipe for human pose, EasyOCR for plate text.
- **Classical / heuristic algorithms**: IoU + centroid tracking, geometric tests (point-in-polygon), color thresholding in HSV space, aspect-ratio/velocity heuristics, cosine similarity matching, and simple linear-regression trend analysis.

This hybrid design keeps the system runnable on commodity hardware (CPU / Apple-Silicon MPS / CUDA) while still providing rich behavioral intelligence.

---

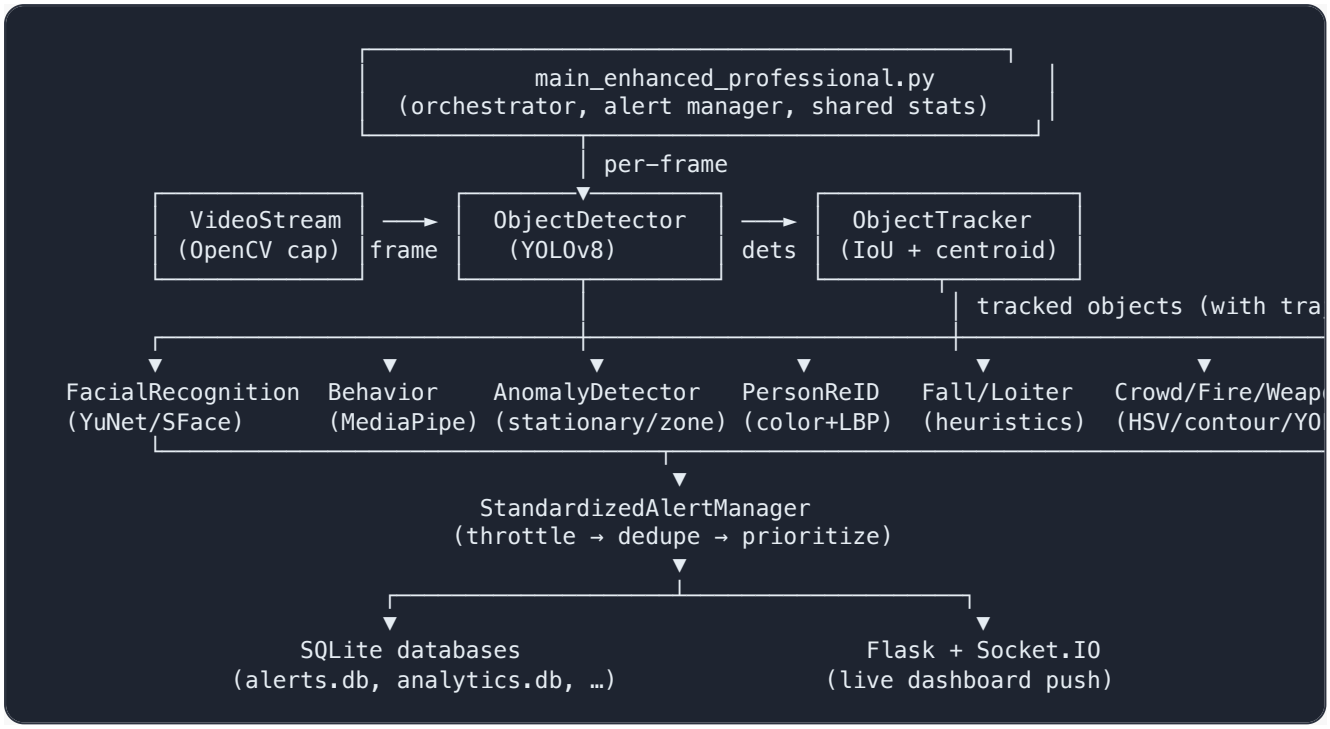
2. Technology Stack

Layer	Library	Role
Video I/O & classical CV	**OpenCV** (`opencv-python ≥ 4.5`)	Capture, color conversion, morphology, contours, drawing
Numerics	**NumPy ≥ 1.20**, **SciPy ≥ 1.7**	Arrays, linear algebra, Euclidean distance
Data	**pandas ≥ 1.3**	Tabular analytics & evaluation reports
Deep detection	**Ultralytics YOLOv8 ≥ 8.0** (PyTorch backend)	Object detection
DL frameworks	**PyTorch ≥ 1.10**, **TorchVision**, **TensorFlow ≥ 2.8**	Model execution / utilities
Face recognition	**OpenCV DNN (YuNet + SFace)** or `face_recognition` **/dlib**	Face detect + 128-D embeddings
Pose	**MediaPipe**	33-landmark body pose
OCR	**EasyOCR**	License-plate text
Metrics/plots	**scikit-learn**, **matplotlib**, **seaborn**	Evaluation, confusion matrix, charts
Web	**Flask ≥ 2.0**, **Flask-Login**, **Flask-SocketIO ≥ 5.0**	Dashboard, auth, real-time push
Alerts	**Twilio ≥ 7.0**	SMS notifications
Config	**python-dotenv**, **PyYAML**	`.env` secrets, dataset YAML

Runtime device selection is automatic with the priority **CUDA (NVIDIA) → MPS (Apple GPU) → CPU**.

---

3. High-Level Architecture & Data Flow



Key design points:

- **Thread-safe shared state**: a global `shared_stats` dict guarded by `shared_stats_lock` (a `threading.Lock`) tracks live counts, daily tallies, and detection-session timing.
- **Dashboard runs in its own thread** via `socketio.run(...)` so video processing never blocks the UI.
- **Graceful shutdown** through `signal_handler` toggling a global `running` flag.

---

4. Project Structure

```
smart_surviance/
├── main_enhanced_professional.py   # Orchestrator + StandardizedAlertManager
├── config/
│   ├── settings.py                 # Runtime constants (sources, thresholds, Twilio, Flask)
│   └── model_config.py             # Training/detection/eval hyper-parameter dictionaries
├── src/
│   ├── video_processing/           # VideoStream, ThreadedVideoStream, pipeline
│   ├── object_detection/           # detector, enhanced_detector, tracker
│   ├── anomaly_detection/          # analyzer (stationary + restricted area)
│   ├── advanced_features/          # 15 feature modules (face, behavior, reid, fall, …)
│   ├── alert_system/               # notifier (templates + history)
│   ├── model_training/             # data_collector, trainer, evaluator
│   └── dashboard/                  # Flask app, routes, templates, static
├── *.db                            # SQLite stores (one per subsystem)
├── known_faces/ captured_faces/    # Face enrollment + auto-captured gallery
├── models/                         # (pretrained / custom YOLO + ONNX face models)
└── logs/                           # Rotating run logs
```

---

5. The Processing Pipeline (Frame Life-Cycle)

For each captured frame `F` the orchestrator's `process_frame_with_all_features` performs, in order:

1. **Detection** — `detections = detector.detect(F)` → list of `{bbox, class_id, class_name, confidence}`.
2. **Live-stat update** — count objects, throttle daily person tally with a 5-second session window.
3. **Tracking** — `tracker.update(detections)` assigns stable IDs and trajectories.
4. **Facial recognition** — detect/identify faces, auto-capture unique snapshots.
5. **Person alerts** — fuse detections with face identity: known → low priority, unknown face → high, else generic.
6. **Behavior analysis** — pose-based activity recognition + crowd metrics.
7. **Anomaly detection** — stationary/unattended + restricted-area violations.
8. **Person Re-ID** — appearance feature extraction & gallery matching.
9. **Alert generation** — every event is funneled through the `StandardizedAlertManager`.

The temporal session-windowing avoids double counting: a person tally only increments when `current_time − last_person_detection > 5.0 s`, and a face recognition tally when the gap `> 10 s`.

---

6. Core Module 1 — Video Acquisition

File: `src/video_processing/video_stream.py`

Two classes:

- `VideoStream` — synchronous capture wrapping `cv2.VideoCapture`.
- `ThreadedVideoStream` — a producer/consumer variant using a background capture thread and a bounded `queue.Queue(maxsize=buffer_size)`. When the buffer is full, the **oldest** frame is dropped (`get_nowait()` then `put_nowait()`), keeping latency low.

Frame-rate control (mathematics)

To cap throughput at a target FPS `f_t`, the reader sleeps for the residual of the frame budget:

```
Δt        = t_now − t_last            (time since last frame)
budget    = 1 / f_t                   (seconds per frame)
sleep     = max(0, budget − Δt)
```

The **measured FPS** is the reciprocal of the inter-frame interval:

```
f_measured = 1 / (t_now − t_last)
```

If a frame read fails, the module synthesizes a "No Camera Feed" test pattern so downstream stages never crash on `None` — a robustness pattern that keeps the pipeline alive during camera dropouts.

Frame resizing to the configured `W×H` (default 640×480) is a simple bilinear interpolation via `cv2.resize`, standardizing tensor shapes for the detector.

---

7. Core Module 2 — Object Detection (YOLOv8)

Files: `src/object_detection/detector.py`, `enhanced_detector.py` **Config:** `config/model_config.py`, `config/settings.py`

`ObjectDetector` is a façade that delegates to the `EnhancedObjectDetector` when available, else to a basic Ultralytics `YOLO` wrapper. Default backbone: `yolov8s.pt`; auto-loads the most recent custom `.pt` from `models/custom/` if present.

7.1 What YOLO computes (theory)

YOLO ("You Only Look Once") is a **single-stage** detector. A fully-convolutional CNN backbone + neck (feature pyramid) produces feature maps at multiple strides; a detection head predicts, per spatial cell and anchor/scale:

- box geometry `(x, y, w, h)`,
- an objectness/confidence score,
- a class-probability vector over the 80 COCO classes.

The final per-box confidence used for thresholding is effectively:

```
score = P(object) · max_c P(class_c | object)
```

Boxes are kept when `score ≥ conf_threshold` (default 0.4 enhanced / 0.5 basic).

7.2 Non-Maximum Suppression (NMS)

Overlapping boxes for the same object are pruned by NMS using the **Intersection-over-Union** metric. For two boxes `A` and `B`:

```
IoU(A,B) = area(A ∩ B) / area(A ∪ B)
```

If `IoU ≥ iou_threshold` (default 0.45), the lower-scoring box is discarded. The system uses **class-aware NMS** (`agnostic_nms = False`) so boxes of different classes don't suppress each other, and allows up to `max_det = 300` detections for crowded scenes.

7.3 Surveillance-specific enhancements

`EnhancedObjectDetector` adds:

- **Configurable inference resolution** `imgsz` (640/960/1280). Higher resolution improves small/distant object recall (more pixels per object) at quadratic compute cost; on CPU it auto-drops 960→640.
- **Optional CLAHE preprocessing** — Contrast-Limited Adaptive Histogram Equalization on the L channel of LAB space, followed by a bilateral filter. CLAHE equalizes local histograms with a clip limit to avoid over-amplifying noise. *(Off by default because it shifts the image away from YOLO's training distribution.)*
- **Class filtering** — restrict outputs to an allow-list, mapping class names → COCO IDs to silence irrelevant classes.
- **Spatial metadata** — each detection is annotated with normalized center, normalized size, area ratio, and a 3×3 spatial zone (`top_left` , `center` , …) determined by thresholds at 0.33 and 0.67 of frame width/height.
- **Edge/size filtering** — tiny objects (`area < 1%` of frame) and edge detections require higher confidence (≥ 0.6) to survive, reducing false positives.

Normalized geometry:

```
center_x = (x1+x2)/2 / W      obj_w = (x2−x1)/W
center_y = (y1+y2)/2 / H      obj_h = (y2−y1)/H
area_ratio = obj_w · obj_h
```

A **running average confidence** is maintained incrementally (Welford-style mean):

```
avg_n = (avg_{n−1} · (n−1) + avg_conf_frame) / n
```

---

8. Core Module 3 — Object Tracking

File: `src/object_detection/tracker.py`

`ObjectTracker` is a lightweight **tracking-by-detection** associator (centroid + IoU), conceptually a simplified SORT without a Kalman filter. State per track: `centroid` , `bbox` , `class_id` , `confidence` , and a `trajectory` list of past centroids.

8.1 Centroid

```
c = ( (x1+x2)//2 , (y1+y2)//2 )
```

8.2 Association cost matrix

For every existing object `i` and new detection `j` a cost `D[i,j]` is built:

- If classes differ → `D[i,j] = ∞` (never match across classes).

- Compute `IoU(box_i, box_j)` and Euclidean centroid distance `d = √((xi−xj)² + (yi−yj)²)`.
- **Hybrid cost**: `D[i,j] = (1 − IoU)·100 if IoU > 0.3 (prioritize box overlap) = d otherwise (fall back to distance)`

This makes well-overlapping boxes extremely cheap to match while still allowing distance-based association for fast-moving or briefly-separated objects.

8.3 Greedy assignment

Rather than the optimal Hungarian algorithm, a **greedy** scheme repeatedly picks the global minimum of `D`, fixes that `(i,j)` pair, then sets that row and column to `∞`. Matches with cost `> max_distance` (default 50) are rejected. Complexity ≈ `O(min(N,M) · N·M)`; fine for the small object counts typical of a single camera.

8.4 Track lifecycle

- Unmatched detections → **registered** as new tracks with a fresh incrementing ID.
- Unmatched tracks → `disappeared[id] += 1`; removed once `disappeared > max_disappeared` (default 30 frames ≈ 1 s at 30 fps). This tolerates short occlusions.
- Trajectories feed downstream movement/loitering/fall logic.

---

9. Core Module 4 — Anomaly Detection

File: `src/anomaly_detection/analyzer.py`

Two classical rule-based detectors driven entirely by track geometry.

9.1 Stationary / unattended object

Per-frame movement between the last two trajectory points:

```
movement = √((x_t − x_{t−1})² + (y_t − y_{t−1})²)
is_stationary = movement < 5 px
```

A counter `stationary_objects[id]` increments while stationary, resets on motion. When it reaches `stationary_threshold` (default 30 frames) **and** the class ∈ {backpack, handbag, suitcase, sports ball, bottle}, an `unattended_object` anomaly is emitted with `frames_stationary` (converted to seconds downstream at 30 fps).

9.2 Restricted-area violation (point-in-polygon)

Restricted zones are polygons (≥ 3 vertices). Membership of a centroid uses OpenCV's `cv2.pointPolygonTest(area, centroid, False)`, which implements the **winding/ray-casting** test: `≥ 0` means inside or on the boundary. The classic ray-casting principle: a point is inside iff a ray to infinity crosses the polygon edges an **odd** number of times.

Class-dependent confidence gating reduces false alarms:

Class	Min confidence to trigger
person	0.5
car / truck / bus / motorcycle / bicycle	0.7
backpack / handbag / suitcase	0.8
anything else	0.9

9.3 Event de-duplication

Before logging, an event is suppressed if an identical `(type, object_id)` event occurred within the last 5 seconds among the last 10 events — a temporal-locality dedupe.

---

10. Advanced Feature — Facial Recognition

File: `src/advanced_features/facial_recognition.py`

A dual-backend system, auto-selecting the best available:

- **OpenCV backend (preferred): YuNet** ONNX detector + **SFace** ONNX recognizer → 128-D embedding. No dlib needed; ideal for Apple Silicon / CPU.
- **dlib backend:** the `face_recognition` library (HOG or CNN detector + 128-D encoding).

10.1 Embeddings and L2 normalization

Each face crop is aligned (`alignCrop`) and mapped to a feature vector $v \in \mathbb{R}^{128}$, then **L2-normalized**:

$$\hat{v} = v / \|v\|_2 \qquad \text{where } \|v\|_2 = \sqrt{\Sigma\, v_i^2}$$

Normalization places all embeddings on the unit hypersphere so that the dot product equals cosine similarity.

10.2 Matching metrics

OpenCV/SFace — cosine similarity (higher = more similar):

$$\cos(a,b) = \hat{a} \cdot \hat{b} = \Sigma\, \hat{a}_i\, \hat{b}_i \qquad \text{(both unit vectors)}$$

A face is recognized as a known identity when $\cos \geq 0.363$ (`SFACE_MATCH_THRESHOLD`). Reported confidence is the similarity itself.

dlib — Euclidean distance with a tolerance:

```
d(a,b) = ‖a − b‖₂
match if d ≤ tolerance (0.6); confidence = 1 − d
```

The best match is `argmax cos` (OpenCV) or `argmin d` (dlib).

10.3 Face quality score

Used to grade enrollment images:

```
sharpness  = Var( Laplacian(gray) )      # focus measure (edge energy)
brightness = mean(gray)
quality    = min(1, (sharpness/1000) · (brightness/255) · 2)
```

The Laplacian variance is the standard "blur metric": sharp images have high-frequency content → high variance; blurry images → low variance.

10.4 Unique-face gallery (dedup)

Auto-captured snapshots are deduplicated by comparing embeddings; a new capture is merged into an existing one if `cos ≥ 0.45` (`SFACE_DEDUP_THRESHOLD`). Unknown faces are additionally hashed (`hash(round(embedding,2).tobytes())`) to count recurrences in the `unknown_faces` table.

Persistence: encodings are pickled (tagged with the backend so cross-backend reuse is avoided) and mirrored into SQLite (`known_faces`, `face_detections`, `unknown_faces`, `captured_faces`).

10.5 Exponential moving average of processing time

```
t_avg ← 0.9 · t_avg + 0.1 · t_frame
```

A standard EMA smoothing latency telemetry with decay 0.9.

---

11. Advanced Feature — Behavior Analysis & Pose Estimation

File: `src/advanced_features/behavior_analysis.py`

Uses **MediaPipe Pose** (33 body landmarks, each with normalized `x,y,z` and `visibility ∈ [0,1]`). The system extracts a subset of 13 key joints (nose, shoulders, elbows, wrists, hips, knees, ankles).

11.1 Derived pose features

- **Body height** ≈ `|y_ankle − y_nose|` (in normalized coordinates).
- **Body width** ≈ `|x_right_shoulder − x_left_shoulder|`.

- **Body lean angle** — the shoulder-line inclination: `θ = atan2(Δy, Δx) · 180/π`
- **Pose stability** — a confidence/presence blend over key joints: `visibility_score = (Σ visibilityⱼ) / |J| presence_score = (#joints with visibility > 0.5) / |J| stability = (visibility_score + presence_score) / 2`

11.2 Activity recognition (speed thresholds)

Movement speed is the centroid displacement between consecutive frames:

```
speed = √((cx_t − cx_{t−1})² + (cy_t − cy_{t−1})²)   [pixels/frame]
```

Rule-based classifier:

Condition	Activity
speed < 5 and body_height < 0.6	sitting
speed < 5	standing
5 ≤ speed < 20	walking
speed ≥ 20	running

(The class also stores reference speed ranges in `behavior_patterns`, e.g. walking 0.5–2.0 m/s, running 2.0–8.0 m/s, for richer rules.)

11.3 Suspicious-behavior heuristics

Flagged when any holds:

- **Loitering**: `standing` and `(now − first_seen) > 300 s`.
- **Unstable pose**: `pose_stability < 0.3`.
- **Erratic activity**: among the last 5 activities, ≥ 4 are distinct (excessive switching).

11.4 Crowd density (frame-level)

```
density = person_count / (H · W) · 10000     [persons per 10k pixels]
```

Severity tiers: `high_density_crowd` if density > 0.01, `large_gathering` if count > 20.

---

12. Advanced Feature — Person Re-Identification

File: `src/advanced_features/person_reid.py`

Matches people across cameras/time using a **hand-crafted appearance descriptor** (no trained ReID network — explicitly a "SimpleFeatureExtractor").

12.1 Feature vector construction

Each person crop is resized to 64×128, then:

- **Color histograms**: per BGR channel, 32-bin histogram → `3 × 32 = 96` values.
- **Local Binary Pattern (LBP)**: for sampled pixels, compare the 8 neighbors to the center; each neighbor contributes a bit: `LBP(c) = Σ_{p=0}^{7} s(I_p − I_c) · 2^p , s(z) = 1 if z>0 else 0` A subset of 100 LBP codes is kept. Final descriptor ≈ 196-D (96 histogram + 100 LBP), L2-normalized.

LBP is a classic **texture descriptor**: it encodes local intensity ordering and is invariant to monotonic illumination changes — useful for clothing texture.

12.2 Similarity & matching

Matching uses cosine similarity:

```
sim(f1,f2) = (f1 · f2) / (‖f1‖ ‖f2‖) ,   clamped to ≥ 0
```

A detection matches the gallery person with the highest `sim` provided `sim > similarity_threshold` (0.7); otherwise a new person ID is created. Cross-camera appearances (a person seen by ≥ 2 cameras) are logged as `cross_camera_matches`.

12.3 Appearance hash & gallery management

A 32×64 crop is MD5-hashed (first 16 hex chars) for a compact appearance fingerprint. When the gallery exceeds `max_gallery_size` (1000), the oldest 10% (by `last_seen`) are archived — an LRU-style eviction.

---

13. Advanced Feature — Fall Detection

File: `src/advanced_features/fall_detection.py`

Detects falls from bounding-box geometry and vertical dynamics (no extra model required).

13.1 Signals

- **Aspect ratio**: `ar = width / height`. Standing humans are tall (`ar < 1`); a fallen person is wide (`ar > 1`).
- **Vertical velocity**: `v_y = cy_t − cy_{t−1}` (positive = downward in image coordinates).
- **Height reduction**: tracked vs. a learned `standing_height` reference (max upright height seen).

13.2 Fall condition

A frame is "fallen" if:

```
is_horizontal   = ar > aspect_ratio_threshold (1.0)
height_reduced  = height < 0.6 · standing_height
rapid_descent   = v_y > velocity_threshold (50 px/frame)

fall_frame = is_horizontal OR (height_reduced AND rapid_descent)
```

13.3 Temporal confirmation & hysteresis

A fall is only **confirmed** after `confirmation_frames` (10) consecutive fall-frames, preventing single-frame glitches. Recovery uses hysteresis: upright frames decrement the counter by 2 each, and the state clears only at 0. A `cooldown_seconds` (30 s) prevents repeated alerts for the same person.

Confidence scales with persistence:

```
confidence = min(1, fallen_frames / (2 · confirmation_frames))
```

---

14. Advanced Feature — Loitering Detection

File: `src/advanced_features/loitering_detection.py`

Tracks **dwell time** within a movement tolerance and escalates through progressive alert levels.

14.1 Displacement & reset

```
displacement = √((cx − cx₀)² + (cy − cy₀)²)
```

If `displacement > movement_tolerance` (50 px) the person is deemed to have left → tracking resets (`first_seen` and anchor position updated). Otherwise dwell time accumulates:

```
dwell = now − first_seen
```

14.2 Progressive thresholds

Dwell time	Level	Priority
≥ 60 s	warning	medium
≥ 180 s	alert	high
≥ 300 s	critical	critical

Alerts fire on **level change** (not every frame) and respect a `alert_cooldown` (60 s). Zone membership uses the same `pointPolygonTest` as the anomaly detector. The visual indicator radius grows with dwell: `r = 30 + (dwell/60)·10`.

---

15. Advanced Feature — Abandoned Object Detection

File: `src/advanced_features/abandoned_object.py`

A more sophisticated unattended-object detector that adds **owner association**.

15.1 Object keying & stationarity

Each suspicious object (bag, suitcase, box, …) is keyed by class + coarse grid cell (`int(cx//50)`, `int(cy//50)`) to give it temporal identity. Stationarity uses displacement vs. a `movement_tolerance` (15 px); a moved object resets its timer.

15.2 Owner logic (the core idea)

1. On first sight, the **nearest person** within `owner_distance_threshold` (200 px) is recorded as owner: `owner = argmin_p √((x_obj − x_p)² + (y_obj − y_p)²)`
2. Once the object has been stationary ≥ `stationary_threshold` (30 s), the system measures owner distance.
3. If the owner moves beyond 200 px (or no person is visible), an **owner-departed timer** starts.
4. After `alert_threshold` (60 s) of departure, an `abandoned_object` HIGH alert fires.

This two-stage temporal logic (stationary → owner departed → grace period) is exactly the model used in real transit-security "left luggage" analytics, reducing false alarms for objects whose owner is merely standing nearby.

---

16. Advanced Feature — Crowd Density & Heatmaps

File: `src/advanced_features/crowd_density.py`

Estimates spatial crowd concentration on a grid (default 16×12) and renders a decaying heatmap.

16.1 Grid binning with Gaussian-like spread

Each person centroid maps to a cell `(gx, gy)`. Mass is deposited not just on that cell but its 3×3 neighborhood with distance-weighted contributions (Manhattan distance):

```
weight = 1.0   if (dx,dy) = (0,0)
       = 0.3   for the 8 neighbors
```

This is a discrete, separable smoothing kernel that prevents harsh single-cell spikes.

16.2 Temporal decay (exponential moving accumulator)

The cumulative heatmap `H` is updated each frame:

```
H ← decay · H + current_grid ,   decay = 0.95
```

This is a **leaky integrator / EMA**: older crowd presence fades geometrically (half-life ≈ `ln(2)/ln(1/0.95) ≈ 13.5` frames), so the heatmap reflects recent persistent congestion.

16.3 Overcrowding & visualization

A zone is overcrowded when its current count `≥ overcrowding_threshold` (10). Alerts are throttled (60 s) and priority escalates to `high` when `person_count > 2 × threshold`. For display, `H` is min-max normalized to `[0,255]`, upscaled bicubically to frame size, colorized with `COLORMAP_JET`, masked (only cells > 10), and alpha-blended at opacity 0.4:

```
out = (1 − α)·frame + α·heatmap_color
```

---

17. Advanced Feature — Fire & Smoke Detection

File: `src/advanced_features/fire_smoke_detection.py`

Classical color/texture analysis with multi-frame verification (no CNN — a fast first-line detector).

17.1 Fire detection (HSV color segmentation)

Convert to HSV and threshold the fire-color band:

```
fire_lower = (H=0,  S=100, V=100)
fire_upper = (H=35, S=255, V=255)
mask = inRange(HSV, fire_lower, fire_upper)
```

Morphological **close then open** (5×5 elliptical kernel) removes holes/specks. Contours with `area > 500` px become candidates. The **fire score** blends color coverage and brightness:

```
fire_percentage = (#fire pixels in box) / box_area
brightness      = mean(ROI)
score = 0.7 · fire_percentage + 0.3 · min(brightness/255, 1)
```

17.2 Smoke detection (texture/grayness)

Smoke is low-saturation, low-contrast, grayish. After Gaussian blur, regions near the mean intensity (`mean ± 30`) are masked. The **smoke score**:

```
texture_score = max(0, 1 − Var(gray)/1000)      # smoke = low variance
gray_score    = max(0, 1 − std(mean_color)/50)  # smoke = low color spread
score = 0.6 · texture_score + 0.4 · gray_score
```

17.3 Multi-frame verification

Detections are pushed into a ring buffer of length `verification_frames` (5). A detection is confirmed only if ≥ 60% of buffered frames exceed sensitivity — a **majority temporal vote** that suppresses flicker. Severity is `(area/k)·confidence` bucketed into LOW/MEDIUM/HIGH/CRITICAL.

> Note: color heuristics will flag fire-colored objects (e.g. bright clothing); a trained CNN is the recommended production upgrade, as the code comments acknowledge.

---

18. Advanced Feature — Weapon Detection

File: `src/advanced_features/weapon_detection.py`

Runs YOLOv8 (currently the base `yolov8s.pt`; intended to be swapped for a fine-tuned weapon model) and filters for weapon-like class names (knife, gun, rifle, pistol, blade, bat, …).

18.1 Threat taxonomy

```
gun/rifle/pistol → CRITICAL (priority 1)
knife/machete    → HIGH     (priority 2)
bat              → MEDIUM   (priority 3)
stick            → LOW      (priority 4)
```

18.2 Weapon→person association

A detected weapon is linked to a person if its center lies inside a person box; otherwise to the nearest person within 150 px:

```
weapon_center = ((x1+x2)/2, (y1+y2)/2)
inside if px1 ≤ cx ≤ px2 and py1 ≤ cy ≤ py2
else associate argmin distance if min_dist < 150 px
```

This containment-then-nearest logic attributes a weapon to its likely holder for actionable alerts.

---

19. Advanced Feature — Violence Detection

File: `src/advanced_features/violence_detection.py`

Analyzes **pairwise interactions** between people using motion statistics (violence usually needs ≥ 2 people).

19.1 Per-person motion features

Per person, a temporal buffer (length 30) stores centroids and speeds:

```
speed_t        = √((cx_t−cx_{t−1})² + (cy_t−cy_{t−1})²)
avg_movement   = mean(speeds)
max_movement   = max(speeds)
movement_var   = Var(speeds)
rapid_movement = max_movement > 0.6 · 100
```

Movement **variance** captures erratic, jerky motion typical of fighting (high variance) vs. steady walking (low variance).

19.2 Pairwise violence score (weighted sum)

For each pair within `interaction_proximity` (100 px):

```
score = 0.30 · [rapid_movement_a OR rapid_movement_b]
      + 0.20 · [avg_variance > 50]
      + 0.30 · mean(aggressive_pose_a, aggressive_pose_b)
      + 0.20 · max(0, 1 − distance/proximity)
score = min(score, 1)
```

This is a **linear opinion pool** of four behavioral cues (rapid motion, motion chaos, aggressive pose, proximity). Threat level buckets at 0.6/0.75/0.9. *(Aggressive-pose scoring is a stub awaiting a pose classifier.)*

---

20. Advanced Feature — PPE Compliance Detection

File: `src/advanced_features/ppe_detection.py`

Checks per-person presence of required safety gear by **region-of-interest color analysis** relative to the person box.

20.1 Region partitioning

Within a person box of height `h`, width `w`:

PPE	ROI
helmet	head: top `h/4`
vest	torso: middle `h/4 … 3h/4`
mask / glasses	face: top `h/3`, central `w/4 … 3w/4`
gloves	hands: bottom `2h/3 … h`

20.2 Color-presence test (HSV)

For colored PPE (hi-vis helmet/vest), the fraction of ROI pixels in a color band must exceed 15%:

```
ratio = countNonZero(inRange(HSV, lower, upper)) / ROI_area
detected if ratio > 0.15
```

20.3 Mask detection via skin exposure

A mask is inferred when the **lower** face shows little skin:

```
skin_mask = inRange(HSV_face, [0,20,70], [20,255,255])
lower_skin_ratio = countNonZero(lower_half) / lower_half_area
mask_present if lower_skin_ratio < 0.3
```

20.4 Compliance rate

Required PPE depends on `zone_type` (construction/hospital/lab…). Compliance:

```
compliant ⇔ |missing_ppe| = 0
compliance_rate = compliant / total_checks · 100
```

---

21. Advanced Feature — License Plate Recognition (ANPR)

File: `src/advanced_features/license_plate_recognition.py`

A two-stage **detect-then-read** ANPR pipeline.

21.1 Plate localization

- **Primary**: Haar cascade (`haarcascade_russian_plate_number.xml`).
- **Fallback (contour geometry)**: bilateral filter → **Canny edges** → contours → polygon approximation (`approxPolyDP`). A region is a plate candidate if it approximates to **4 corners** and the aspect ratio is plate-like: `2.0 ≤ w/h ≤ 5.0 , w > 50 , h > 15`

Canny uses gradient magnitude + hysteresis thresholding (here 30/200) to find strong edges.

21.2 Preprocessing for OCR

Grayscale → bilateral filter → **adaptive Gaussian threshold** (local binarization robust to uneven lighting) → morphological close. Adaptive thresholding computes a per-pixel threshold from a Gaussian- weighted neighborhood, ideal for plates under variable illumination.

21.3 Recognition & validation

EasyOCR returns `(bbox, text, confidence)` per region; the highest-confidence string is kept above the threshold. Text is cleaned (`[^A-Z0-9]` removed) and validated:

```
valid ⇔ 4 ≤ len ≤ 10  AND  has_letter  AND  has_digit
```

Validated plates are checked against **whitelist** (green) / **blacklist** (red) sets loaded from SQLite, and associated with a vehicle if the plate center lies inside a vehicle box.

---

22. Real-Time Analytics Engine

File: `src/advanced_features/real_time_analytics.py`

A background thread (interval `update_interval`, default 60 s) maintaining metric/event buffers and computing statistics, trends, and predictions over a sliding `analysis_window` (1 hour).

22.1 Descriptive statistics

Per metric: current, mean, median, min, max, and **sample standard deviation**:

$$\sigma = \sqrt{\frac{1}{n-1}\sum (x_i - \bar{x})^2}$$

22.2 Trend via least-squares slope

Trend direction comes from the **ordinary least squares** slope over the value series $x_i = i$:

$$\text{slope} = \frac{n\cdot\Sigma(i\cdot y_i) - \Sigma i\cdot\Sigma y_i}{n\cdot\Sigma i^2 - (\Sigma i)^2}$$

Classified as increasing/decreasing/stable against a data-scaled threshold $0.1\cdot(\max-\min)/n$.

22.3 Prediction (linear extrapolation)

Next value is predicted from a 5-point regression line $\hat{y} = \text{slope}\cdot x + \text{intercept}$ using mean-centered sums:

```
slope = Σ(xᵢ−x̄)(yᵢ−ȳ) / Σ(xᵢ−x̄)²
ŷ_{next} = slope·n + intercept   (clamped ≥ 0)
```

22.4 Event-frequency analysis

For each event type, inter-arrival intervals `Δtᵢ` are computed; the hourly frequency is `3600 / mean(Δt)`. A 20% shift in recent vs. older mean interval flags increasing/decreasing frequency. Volatility is the coefficient of variation `σ/mean`. Activity levels are bucketed (inactive/low/moderate/high/very_high) by event counts.

---

23. Alert System & Throttling Mathematics

Files: `main_enhanced_professional.py` (`StandardizedAlertManager`), `src/alert_system/notifier.py`

23.1 Standardized alert manager

Every subsystem funnels events here. Each alert gets an ID, type, message, priority (low/medium/high), timestamp, and JSON payload, then is persisted to `alerts.db` and pushed to the dashboard via `socketio.emit('new_alert', …)`.

23.2 Type-aware throttling (noise control)

A per-type minimum inter-alert interval suppresses spam. With key `k = type + message[:50]`:

```
if (now − last_alert[k]) < throttle_time[type]:  drop
else: emit and set last_alert[k] = now
```

Type	Throttle (s)
intrusion	5
object_detection	10
suspicious_behavior	20
unknown_person	30
unattended_object	60
system_status	60
restricted_area_violation	180

This is effectively a **token-bucket of size 1 with a fixed refill period per alert key** — guaranteeing at most one alert of a given kind per window.

23.3 Priority assignment logic

Priorities are derived contextually, e.g. unattended objects become **high** when `frames_stationary > 300` (≈ 10 s at 30 fps); restricted-area violations are high only for `person`. History is capped at the last 1000 alerts (bounded memory).

23.4 Template-based notifier

`AlertSystem` formats messages from per-type templates, keeps a bounded history, supports acknowledgment, and can append JSONL to a log file. Frame counts convert to seconds at 30 fps: `duration = frames_stationary / 30`.

---

24. Model Training & Evaluation

Files: `src/model_training/trainer.py`, `evaluator.py`, `data_collector.py`, `config/model_config.py`

24.1 Custom training (`CustomModelTrainer`)

Wraps Ultralytics training. Builds the YOLO dataset layout (`images/{train,val}`, `labels/{train,val}`) and a `data.yaml`; can **auto-split** train/val (default 20% val) and **auto-annotate** images using a pretrained model (writing normalized YOLO boxes).

YOLO label format (normalized):

```
class_id  center_x/W  center_y/H  width/W  height/H
```

Hyper-parameters (from `model_config.py` / `create_training_config`): 100 epochs, batch 16, imgsz 640, `lr0 = 0.01`, `lrf = 0.01`, momentum 0.937, weight decay 5e-4, 3 warmup epochs; loss gains box 0.05 / cls 0.5 / dfl 1.5; AMP (mixed precision) on. Augmentations are surveillance-tuned: HSV jitter, translate 0.1, scale 0.5, horizontal flip 0.5, mosaic 1.0; rotation/shear/perspective/vertical-flip disabled (cameras are fixed and upright).

24.2 Evaluation metrics (`ModelEvaluator`)

Computes the standard detection metrics:

- **Precision** `P = TP / (TP + FP)`
- **Recall** `R = TP / (TP + FN)`
- **F1** `= 2PR / (P + R)`

- **mAP@0.5** — mean Average Precision at IoU 0.5; AP is the area under the precision–recall curve, `AP = ∫₀¹ P(R) dR`, averaged over classes.
- **mAP@0.5:0.95** — AP averaged over IoU thresholds 0.5…0.95 (COCO primary metric).

A "true positive" requires both correct class and `IoU(pred, gt) ≥ iou_threshold`. The evaluator emits JSON + human-readable reports, per-class bars, P/R/F1 charts, and multi-model comparison tables (best model = max mAP@0.5:0.95; fastest = min inference time).

---

25. Web Dashboard

Files: `src/dashboard/__init__.py`, `app.py`, `routes/*.py`, `templates/*.html`

A Flask **application-factory** (`create_app`) registering three blueprints:

- `main_routes` — pages (home, dashboard, feature detail).
- `auth_routes` — login (Flask-Login).
- `api_routes` (prefix `/api`) — JSON endpoints + `feature_data`.

Real-time updates use **Flask-SocketIO** (`cors_allowed_origins="*"`): the orchestrator emits `new_alert` events that the browser receives over WebSockets without polling. The server binds to `127.0.0.1:8082` (localhost, required for browser camera-access permissions). Config: 16 MB upload cap, secret key from env, template auto-reload.

---

26. Persistence Layer (Databases)

Each subsystem owns a dedicated **SQLite** database, giving clean separation and simple per-feature querying:

Database	Owner	Key tables
`alerts.db`	Alert manager	`alerts`
`analytics.db`	Analytics engine	`realtime_metrics`, `activity_patterns`, `analytics_insights`, `performance_benchmarks`
`face_database.db`	Facial recognition	`known_faces`, `face_detections`, `unknown_faces`, `captured_faces`
`behavior_analysis.db`	Behavior	`behavior_events`, `pose_data`, `crowd_events`
`person_reid.db`	Re-ID	`person_gallery`, `person_tracks`, `cross_camera_matches`, `camera_info`
`fall_detection.db`	Fall	`fall_events`
`loitering_detection.db`	Loitering	`loitering_events`
`abandoned_objects.db`	Abandoned obj	`abandoned_object_events`
`crowd_density.db`	Crowd	`density_snapshots`, `overcrowding_events`
`fire_smoke_detection.db`	Fire/Smoke	`fire_smoke_events`
`weapon_detection.db`	Weapon	`weapon_detections`, `threat_events`
`violence_detection.db`	Violence	`violence_events`, `person_involvement`
`ppe_detection.db`	PPE	`ppe_violations`, `ppe_compliance_log`
`license_plates.db`	ANPR	`plate_detections`, `whitelist`, `blacklist`

Binary artifacts: `face_encodings.pkl` (face embeddings), `person_features.pkl` (Re-ID gallery).

---

27. Consolidated Mathematical Glossary

Concept	Formula	Used in
Centroid	`((x1+x2)/2, (y1+y2)/2)`	tracking, all heuristics
Euclidean distance	`√(Δx² + Δy²)`	tracking, loitering, fall, abandoned, violence
IoU	`area(A∩B) / area(A∪B)`	NMS, tracking, eval matching
Detection score	`P(obj)·max_c P(c|obj)`	YOLO
L2 norm	`√(Σ vᵢ²)`	embedding normalization
Cosine similarity	`(a·b)/(‖a‖‖b‖)`	face match, Re-ID
Euclidean face distance	`‖a−b‖₂`, match if ≤ 0.6	dlib face
Laplacian variance	`Var(∇²I)`	face sharpness/quality
LBP code	`Σ s(I_p−I_c)·2^p`	Re-ID texture
Aspect ratio	`w/h`	fall detection
Vertical velocity	`cy_t − cy_{t−1}`	fall detection
EMA / leaky integrator	`S ← α·S + (1−α)·x`	latency, crowd heatmap
Density	`count/(H·W)·10⁴`	crowd
OLS slope	`(nΣxy − ΣxΣy)/(nΣx² − (Σx)²)`	analytics trend/prediction
Std deviation	`√(Σ(xᵢ−x̄)²/(n−1))`	analytics
Point-in-polygon	ray-casting / `pointPolygonTest`	restricted area, zones
Precision / Recall / F1	`TP/(TP+FP)`, `TP/(TP+FN)`, `2PR/(P+R)`	evaluation
Average Precision	`∫₀¹ P(R) dR`	mAP
Weighted score fusion	`Σ wᵢ·featureᵢ`	violence, fire/smoke score
HSV color masking	`inRange(HSV, lo, hi)`	fire, PPE

28. Limitations & Improvement Opportunities

Strengths. Highly modular; degrades gracefully when optional deps (dlib, MediaPipe, EasyOCR) are missing; mixes fast heuristics with learned models; per-feature persistence; real-time dashboard; device-agnostic (CPU/MPS/CUDA); thoughtful alert de-noising.

Limitations & upgrades.

1. **Heuristic detectors** for fire/smoke, weapons, violence, and PPE rely on color/geometry and will produce false positives. Production should use **dedicated fine-tuned CNNs** (the code comments say as much; weapon detection currently runs the generic `yolov8s.pt`).
2. **Tracker** is greedy IoU+centroid without motion prediction. A **Kalman filter + Hungarian assignment** (full SORT/Deep SORT) would handle occlusion and ID switches better.
3. **Person Re-ID** uses hand-crafted color+LBP features. A trained **OSNet/ResNet ReID embedding** would be far more discriminative across lighting/pose.
4. **Speed in pixels/frame** conflates real-world speed with camera distance/zoom. **Homography/camera calibration** would yield metric speeds for behavior/fall thresholds.
5. **Per-feature SQLite files** could be unified or moved to a server DB for multi-process safety; many modules open a new connection per write (works but not optimal under load).
6. **Security**: the dashboard secret key has a hard-coded fallback and Socket.IO uses `cors_allowed_origins="*"`; Twilio creds live in `settings.py` / `.env`. For deployment, enforce env-only secrets, restrict CORS, and add auth on all API routes.
7. **No unit/integration tests** are present for the analytics math or detectors; adding them would lock in correctness of the thresholds and formulas above.

End of report.